

Understanding the MDT Changes in Duranta (OAI 5G Code)

A beginner-friendly walkthrough of the code and what it does

What is MDT?

MDT stands for Minimization of Drive Tests. Normally, network engineers drive around cities with special equipment to measure signal strength in different areas. MDT lets the phone itself collect and report this information automatically, so engineers don't always need to physically drive around testing signal quality.

1. The Big Picture — What Was Built

The code changes add an MDT feature into OpenAirInterface's (OAI) 5G RRC (Radio Resource Control) code, using the Duranta codebase. Instead of inventing a brand-new message format, the changes reuse the existing MeasurementReport mechanism that 5G networks already use.

In one sentence, the whole system works like this:

The Flow in Plain English

The phone (UE) logs signal strength → the phone packages that into a report → the phone sends the report to the tower (gNB) → the tower saves the report for that specific user.

Here are the key terms used throughout this document:

- **UE (User Equipment):** the phone or device side of the connection.
- **gNB:** the 5G base station / tower.
- **RRC (Radio Resource Control):** the protocol layer that manages the connection setup and control messages between phone and tower.
- **RSRP:** Reference Signal Received Power — a number (in dBm) that tells you how strong the signal is. Less negative = stronger signal.
- **SSB:** Synchronization Signal Block — a basic reference signal used to detect and measure a cell.
- **CSI-RS:** Channel State Information Reference Signal — another, more detailed reference signal used for channel measurements.
- **SRB1:** Signaling Radio Bearer 1 — the communication channel used to carry control messages like MeasurementReports.
- **PDCP:** Packet Data Convergence Protocol — a protocol layer that helps deliver packets reliably between phone and tower.
- **ASN.1:** a standard format used to encode 5G protocol messages so both phone and tower understand them the same way.

2. Phone-Side (UE) Data Structures

These structures define how the phone stores MDT information in memory. They live in `nr_rrc_defs.h` and `rrc_defs.h`.

2.1 The Buffer Size

```
#define NR_MDT_LOG_BUFFER_SIZE 64
```

This line simply says: the phone can store up to 64 signal samples before it needs to send them off or start overwriting old ones. Think of it like a notebook with exactly 64 pages — once it's full, older pages get reused.

2.2 Why a Sample Was Recorded (the trigger reason)

```
typedef enum {  
    NR_MDT_TRIGGER_NONE = 0,  
    NR_MDT_TRIGGER_ENABLE,  
    NR_MDT_TRIGGER_RSRP_DROP,  
    NR_MDT_TRIGGER_LOW_RSRP,  
    NR_MDT_TRIGGER_MEAS_UPDATE,  
    NR_MDT_TRIGGER_PERIODIC,  
} nr_mdt_trigger_reason_t;
```

Every time the phone saves a sample, it also records why it saved it. This is just a labeled list of possible reasons:

- **ENABLE** — MDT logging was just turned on.
- **RSRP_DROP** — the signal suddenly dropped by a noticeable amount.
- **LOW_RSRP** — the signal became too weak.
- **MEAS_UPDATE** — a regular measurement update happened.
- **PERIODIC** — a scheduled, timed check.

2.3 One Measurement Sample

```
typedef struct nr_mdt_sample_s {  
    uint32_t frame;  
    uint16_t hfn;  
    uint8_t gnb_index;  
    uint16_t Nid_cell;  
    bool is_csi_meas;  
    bool is_neighboring_cell;  
    int rsrp_dBm;  
    int delta_rsrp_dB;  
    nr_mdt_trigger_reason_t reason;  
} nr_mdt_sample_t;
```

This structure is one complete signal event. Each field means:

- **frame**: the radio frame number — a way of marking 'when' in radio time.
- **hfn (Hyper Frame Number)**: a longer-running time counter, used alongside the frame number.
- **gnb_index**: which tower the phone is talking to.
- **Nid_cell**: the ID of the radio cell being measured.
- **is_csi_meas**: true if this measurement came from CSI-RS instead of SSB.
- **is_neighboring_cell**: true if this measurement is about a nearby cell rather than the phone's current (serving) cell.

- **rsrp_dBm**: the actual signal strength measured, in dBm.
- **delta_rsrp_dB**: how much the signal changed compared to the last reading.
- **reason**: why this sample was recorded (using the list from section 2.2).

2.4 The Full MDT Control Panel for the Phone

```
typedef struct nr_mdt_state_s {
    bool enabled;
    bool logging_active;
    bool config_received;
    int trigger_drop_threshold_db;
    int low_rsrp_threshold_db;
    int last_rsrp_dBm;
    bool last_rsrp_valid;
    uint32_t sample_count;
    uint32_t flush_count;
    uint32_t next_index;
    nr_mdt_sample_t samples[NR_MDT_LOG_BUFFER_SIZE];
} nr_mdt_state_t;
```

This is the phone's overall MDT 'dashboard' — everything it needs to know about its own MDT logging:

- **enabled**: is MDT turned on right now?
- **logging_active**: is logging currently happening?
- **config_received**: has the phone already received MDT settings from the network?
- **trigger_drop_threshold_db**: how many dB the signal must fall before it counts as a 'drop'.
- **low_rsrp_threshold_db**: the signal level considered 'too weak'.
- **last_rsrp_dBm / last_rsrp_valid**: the previous signal reading, and whether it's usable yet.
- **sample_count**: how many samples are currently stored.
- **flush_count**: how many times the buffer has been emptied and sent.
- **next_index**: where the next sample will be written in the buffer.
- **samples[64]**: the actual storage area for up to 64 samples.

3. Phone-Side (UE) Functions

These functions live in `rrc_UE.c` and implement the phone's MDT behavior in four steps: Enable → Classify → Store → Flush.

3.1 Turning a Reason Code into Readable Text

```
static const char *nr_mdt_reason_to_string(nr_mdt_trigger_reason_t reason)
{
    switch (reason) {
        case NR_MDT_TRIGGER_ENABLE:      return "enable";
        case NR_MDT_TRIGGER_RSRP_DROP:   return "rsrp_drop";
        case NR_MDT_TRIGGER_LOW_RSRP:    return "low_rsrp";
        case NR_MDT_TRIGGER_MEAS_UPDATE: return "meas_update";
        case NR_MDT_TRIGGER_PERIODIC:    return "periodic";
        default: return "none";
    }
}
```

This is just a translator for log messages. Instead of printing a plain number like 3, the logs print a readable word like "low_rsrp", which makes debugging much easier.

3.2 Resetting MDT to Default Settings

```
static void nr_rrc_mdt_reset(rrcPerNB_t *rrcNB)
{
    memset(&rrcNB->mdt, 0, sizeof(rrcNB->mdt));
    rrcNB->mdt.trigger_drop_threshold_db = 3;
    rrcNB->mdt.low_rsrp_threshold_db = -100;
}
```

This function wipes the MDT memory clean and sets two default rules:

- A drop of 3 dB or more counts as a 'signal drop' event.
- A signal at or below -100 dBm counts as 'too weak'.

Example: if the signal goes from -95 dBm to -99 dBm, that's a 4 dB drop, which crosses the 3 dB threshold. If it later reaches -103 dBm, that also crosses the -100 dBm 'too weak' threshold.

3.3 Turning MDT On

```
static void nr_rrc_mdt_enable(rrcPerNB_t *rrcNB, instance_t ue_id, int gnb_index)
{
    if (!rrcNB->mdt.config_received) {
        nr_rrc_mdt_reset(rrcNB);
        rrcNB->mdt.config_received = true;
    }

    rrcNB->mdt.enabled = true;
    rrcNB->mdt.logging_active = true;
    LOG_I(NR_RRC, "[UE %ld][gNB %d] MDT logging enabled ...");
}
```

When the network tells the phone to start MDT, this function runs. If this is the first time, it resets the state to defaults. Then it marks MDT as enabled and actively logging, and prints a confirmation log line.

3.4 Saving One Sample into the Buffer

```
static void nr_rrc_mdt_store_sample(..., int rsrp_dBm, int delta_rsrp_dB,
                                   nr_mdt_trigger_reason_t reason)
{
    nr_mdt_state_t *mdt = &rrcNB->mdt;
    if (!mdt->enabled) return;

    nr_mdt_sample_t *sample = &mdt->samples[mdt->next_index];
    sample->frame = (uint32_t)rrc->current_frame;
    sample->hfn = (uint16_t)rrc->current_hfn;
    sample->rsrp_dBm = rsrp_dBm;
    sample->delta_rsrp_dB = delta_rsrp_dB;
    sample->reason = reason;
    ... (other fields filled in too)

    mdt->next_index = (mdt->next_index + 1) % NR_MDT_LOG_BUFFER_SIZE;
    if (mdt->sample_count < NR_MDT_LOG_BUFFER_SIZE)
        mdt->sample_count++;

    if (mdt->sample_count == NR_MDT_LOG_BUFFER_SIZE) {
        LOG_I(NR_RRC, "buffer full, flushing logged samples");
    }
}
```

```
}  
}
```

Step by step, this function:

- Checks whether MDT is enabled — if not, it does nothing.
- Writes the new sample into the next open slot.
- Moves the 'next slot' pointer forward, wrapping back to 0 after slot 63 (this is called a ring buffer).
- Keeps count of how many samples are currently stored (maxing out at 64).
- Logs a message, and if the buffer just became completely full, it notes that too.

What is a Ring Buffer?

A ring buffer is a fixed-size list that wraps around. Once you fill slot 63, the next new item goes back into slot 0, overwriting the oldest entry. This avoids the memory constantly growing bigger and bigger.

3.5 Sending the Buffer as Reports

```
static void nr_rrc_mdt_flush_report(rrcPerNB_t *rrcNB, instance_t ue_id,  
                                   int gnb_index, const char *reason)  
{  
    nr_mdt_state_t *mdt = &rrcNB->mdt;  
    if (!mdt->enabled || mdt->sample_count == 0) return;  
  
    uint32_t start = (mdt->next_index + NR_MDT_LOG_BUFFER_SIZE  
                     - mdt->sample_count) % NR_MDT_LOG_BUFFER_SIZE;  
    for (uint32_t i = 0; i < mdt->sample_count; i++) {  
        nr_mdt_sample_t *sample = &mdt->samples[(start + i) % NR_MDT_LOG_BUFFER_SIZE];  
        uint8_t buffer[NR_RRC_BUF_SIZE];  
        ...  
        const uint8_t size = do_nrMDTMeasurementReport_SA(..., buffer, sizeof(buffer));  
  
        nr_pdc_data_req_srb(ue_id, /*srb_id=*/1, 0, size, buffer, deliver_pdu_srb_rlc, NULL);  
    }  
  
    mdt->flush_count++;  
    mdt->sample_count = 0;  
    mdt->next_index = 0;  
}
```

This is one of the most important functions — it's how stored samples actually leave the phone and reach the tower. What it does:

- If MDT is off or there's nothing stored, it stops immediately.
- It works out where in the ring buffer the oldest sample sits.
- For every stored sample, it converts the signal reading into the ASN.1 message format using `do_nrMDTMeasurementReport_SA()`.
- It sends the encoded message over SRB1 (the signaling channel), using `nr_pdc_data_req_srb()`.
- After all samples are sent, it resets the sample count and buffer position back to zero, ready to start collecting again.

Note that this loop sends samples one at a time, not as one big combined message — each one goes out as its own standard report.

4. Packaging the Report: asn1_msg.c

```
int do_nrMDTMeasurementReport_SA(long trigger_to_measid,
                                long trigger_quantity,
                                long rs_type,
                                uint16_t Nid_cell,
                                int rsrp_index,
                                bool neighbor_cell_valid,
                                uint16_t neighbor_Nid_cell,
                                int neighbor_rsrp_index,
                                uint8_t *buffer,
                                size_t buffer_size)
{
    return do_nrMeasurementReport_SA(trigger_to_measid, trigger_quantity, rs_type,
                                     Nid_cell, rsrp_index, neighbor_cell_valid,
                                     neighbor_Nid_cell, neighbor_rsrp_index,
                                     buffer, buffer_size);
}
```

This function has an MDT-specific name, but if you look closely, all it does is forward every argument straight into the already-existing `do_nrMeasurementReport_SA()` function — the same one used for normal (non-MDT) measurement reports.

Key Takeaway

There is no brand-new MDT message type yet. MDT currently 'rides along' on the standard NR MeasurementReport message format. This function is a named entry point for MDT, but the underlying report structure is not MDT-specific.

5. Tower-Side (gNB) Data Structures

These live in `nr_rrc_defs.h` and describe how the tower stores a decoded report once it receives one from the phone.

```
typedef struct nr_mdt_report_s {
    time_t received_time;
    uint32_t ue_rrc_id;
    NR_MeasId_t measId;
    uint16_t serving_cell_id;
    int serving_rsrp_dBm;
    int serving_rsrp_index;
    uint16_t neighbor_cell_id;
    int neighbor_rsrp_dBm;
    int neighbor_rsrp_index;
    bool has_neighbor;
} nr_mdt_report_t;
```

- **received_time**: when the tower received this report.
- **ue_rrc_id**: which specific phone/user this report belongs to.
- **measId**: the measurement ID from the RRC configuration.
- **serving_cell_id / serving_rsrp_dBm / serving_rsrp_index**: information about the phone's main current cell.
- **neighbor_cell_id / neighbor_rsrp_dBm / neighbor_rsrp_index**: information about a nearby cell, if one was reported.
- **has_neighbor**: whether neighbor cell data was actually present in this report.

Note: the phone's sample structure (Section 2.3) and the tower's report structure here are related but not identical. The phone's sample is local logging memory before sending; the tower's report is what gets stored after decoding a received message.

6. Preparing Storage for Each New Phone

File: rrc_gNB_ue_context.c

```
rrc_gNB_ue_context_t *rrc_gNB_allocate_new_ue_context(gNB_RRC_INST *rrc_instance_pP)
{
    rrc_gNB_ue_context_t *new_p = calloc(1, sizeof(*new_p));
    if (new_p == NULL) {
        LOG_E(NR_RRC, "Cannot allocate new ue context\n");
        return NULL;
    }
    new_p->ue_context.rrc_ue_id = uid_linear_allocator_new(&rrc_instance_pP->uid_allocator) + 1;
    seq_arr_init(&new_p->ue_context.mdt_reports, sizeof(nr_mdt_report_t));
    rrc_gNB_ue_context_update_time(new_p);

    return(new_p);
}
```

Every time a new phone connects to the tower, the tower creates a fresh 'UE context' — its own private notebook for that phone. The important new line here is:

```
seq_arr_init(&new_p->ue_context.mdt_reports, sizeof(nr_mdt_report_t));
```

This sets up an empty, growable list (called a sequence array) ready to hold MDT reports for this specific phone. So from the very first moment a phone connects, the tower is ready to start collecting its MDT history.

7. Receiving and Storing a Report: store_mdt_report()

File: rrc_gnb.c — this is the tower-side function that runs whenever a MeasurementReport message arrives from a phone.

7.1 Safety Checks First

```
static void store_mdt_report(gNB_RRC_UE_t *ue_ctxt, const NR_MeasurementReport_t *measurementReport)
{
    if (measurementReport == NULL
        || measurementReport->criticalExtensions.present !=
            NR_MeasurementReport__criticalExtensions_PR_measurementReport
        || measurementReport->criticalExtensions.choice.measurementReport == NULL) {
        return;
    }
    ...
}
```

Before doing anything else, the function checks that the incoming message actually is a valid MeasurementReport. If it's not, the function simply exits — this is just defensive programming to avoid crashing on bad data.

7.2 Reading the Serving Cell Signal

```
if (results->measResultServingMOList.list.count > 0) {
    const NR_MeasResultServMO_t *serv = results->measResultServingMOList.list.array[0];
    if (serv->measResultServingCell.physCellId) {
        report.serving_cell_id = *serv->measResultServingCell.physCellId;
    }
}
```

```

}
if (cellResults->resultsSSB_Cell && cellResults->resultsSSB_Cell->rsrp) {
    report.serving_rsrp_index = *cellResults->resultsSSB_Cell->rsrp;
    report.serving_rsrp_dBm = report.serving_rsrp_index + 157;
}
// same idea repeated for CSI-RS measurements
}

```

This section pulls out the serving (main) cell's ID and signal strength. Signal strength in the raw message is stored as a compact index number rather than the real dBm value, so the code converts it back using index + 157. The important idea here isn't the exact math — it's that the raw protocol number gets turned into a human-readable dBm value for storage and logging.

7.3 Reading the Neighbor Cell Signal (if present)

```

if (results->measResultNeighCells &&
    results->measResultNeighCells->present ==
    NR_MeasResults__measResultNeighCells_PR_measResultListNR &&
    results->measResultNeighCells->choice.measResultListNR->list.count > 0) {
    const NR_MeasResultNR_t *neigh = results->measResultNeighCells->choice.measResultListNR-
>list.array[0];
    if (neigh->physCellId) {
        report.neighbor_cell_id = *neigh->physCellId;
        report.has_neighbor = true;
    }
    // RSRP extraction, same pattern as the serving cell
}

```

If the phone also reported a nearby (neighbor) cell, this block extracts that cell's ID and signal strength the same way.

A Current Limitation

This code only looks at the first serving result and the first neighbor result in the list. If a report contained several neighbor cells, only the first one gets saved — the rest are currently ignored.

7.4 Saving the Report and Logging It

```

seq_arr_push_back(&ue_ctxt->mdt_reports, &report, sizeof(report));
ue_ctxt->mdt_report_count++;

LOG_I(NR_RRC,
    "[MDT][gNB UE %d] stored report #%u measId=%d serving_cell=%u "
    "serving_RSRP=%d dBm neighbor_cell=%u neighbor_RSRP=%d dBm\n",
    ...);

```

Finally, the completed report is appended to that phone's personal history list, the report counter goes up by one, and a log line is printed confirming exactly what was stored. This log line is very useful for debugging — you can literally watch reports arrive in the console.

8. What the Configuration File Is For

File: gnb.sa.band78.fr.106prb.pci0.rfsim.conf

This file is not MDT logic — it's the setup file for the simulated radio environment the tower runs in. It's included here because it defines the network the MDT code operates on top of.

- **gNB_ID / gNB_name:** the identity of this simulated base station.

- **tracking_area_code / plmn_list**: network identity settings (operator, region).
- **physCellId / nr_cellid**: the identifiers for the radio cell.
- **dl_frequencyBand / ul_frequencyBand = 78**: this simulation uses 5G band n78.
- **dl_carrierBandwidth / ul_carrierBandwidth = 106**: the amount of radio spectrum used (106 resource blocks).
- **do_CSIRS = 1**: CSI-RS measurements are turned on, which matters because the MDT code checks for CSI-RS results.
- **do_SRS = "periodic"**: the sounding reference signal runs on a regular schedule.

In short: this file sets the stage (the simulated radio environment), while the C code covered earlier is the actual actor performing the MDT logic on that stage.

9. A Simple Worked Example

To make everything concrete, here's a step-by-step walkthrough of one full cycle:

- The network enables MDT on the phone. The phone resets its MDT state and sets thresholds: 3 dB for a drop, -100 dBm for 'too weak'.
- The signal weakens from -96 dBm to -101 dBm — a 5 dB drop that also crosses the -100 dBm threshold.
- The phone stores a new sample in its ring buffer: RSRP = -101 dBm, delta = -5 dB, reason = a drop/low-signal trigger.
- Later, `nr_rrc_mdt_flush_report()` runs. It converts the sample into the standard MeasurementReport ASN.1 format using `do_nrMDTMeasurementReport_SA()`.
- The encoded message is sent over SRB1 using `nr_pdcp_data_req_srb()`.
- The tower receives the message and runs `store_mdt_report()`. It decodes the serving cell's signal value, checks for a neighbor cell, and builds an `nr_mdt_report_t`.
- The report is appended to that phone's `mdt_reports` history list, the counter increases, and a confirmation log line is printed.

10. What Works Today, and What's Still Missing

10.1 What Is Working

- The phone can be enabled for MDT logging and track configurable thresholds.
- Signal samples are stored efficiently using a 64-entry ring buffer.
- Samples are converted and sent using the existing, standards-based MeasurementReport encoder.
- The tower creates dedicated MDT storage for every connected phone.
- Incoming reports are validated, decoded, and saved into a per-phone history, with clear logging.

10.2 What Is Still Incomplete

- There is no dedicated MDT ASN.1 message type yet — MDT currently reuses the standard MeasurementReport format rather than having its own purpose-built structure.
- The tower only stores the first serving result and the first neighbor result per report, not a full list of all measured cells.

- Reports are kept only in memory on the tower — there's no export to a file, database, or analytics system yet.

Bottom Line

This is a working first version that proves the entire MDT path end-to-end: enable → measure → store → send → receive → store. It is best understood as a solid functional prototype rather than a complete, standards-final MDT feature.

11. File-by-File Quick Reference

File	What It Contains	Why It Matters
rrc_defs.h / nr_rrc_defs.h	MDT data structures for the phone (buffer size, sample struct, state struct) and the report struct for the tower	Defines the 'shape' of MDT data on both sides
rrc_UE.c	Phone-side functions: reset, enable, store sample, flush/send report	This is where the phone actually collects and sends MDT data
asn1_msg.c	do_nrMDTMeasurementReport_SA(), which wraps the standard report encoder	Packages a sample into a message the tower can understand
rrc_gNB_ue_context.c	Sets up empty MDT report storage whenever a new phone connects	Makes sure every phone has a place to store its MDT history
rrc_gnb.c	store_mdt_report(), which validates, decodes, and saves incoming reports	This is where the tower actually processes and keeps MDT data
gnb.sa.band78.fr.106prb.pci 0.rfsim.conf	Simulated radio environment settings (band, bandwidth, cell ID, etc.)	Provides the network setup the MDT code runs on top of